Modern University for Technology & Information

Faculty of Computers and Artificial Intelligence

Project Title: Numerical Analysis Solver

Course: (**BS-252)** Numerical Analysis

Supervised by: Dr. Tarek Soliman

Teaching Assistant's : TA. Nada El Sayed

TA. Aya Hesham

Submitted By: Mohamed Omar Aboghalon

ID: 107782

Group: G7

Contents

Chapter 1 : Introduction 3
1. Introduction 3
2. Objectives 3
3. Tools & Technologies Used 4
   1. Programming Language 4
   2. GUI Framework 4
   3. Libraries 4
   4. IDE 4
   5. Other tools 4
Chapter 2 : Application User Guide 5
Chapter 3 : Methods Implementation & Results 6
1. Chapter 1 : FINDING ROOTS OF POLYNOMIALS 6
   1. Bisection Method 6
   2. False position Method 8
   3. Newton'sMethod 10
   4. Simple Fixed point Method 12
   5. Secant Method 14
Chapter 2 : LINEAR ALGEBRAIC EQUATIONS 16
   1. Gauss Elimination 16
   2. LU Decomposition 18
   3. Cramer’s Rule 21
   4. Gauss-Jordan 22

Chapter 1 : Introduction

1. Introduction

Numerical analysis is a fundamental branch of mathematics and computer science that focuses on developing, analyzing, and applying algorithms to obtain approximate solutions for complex mathematical problems. In many real-world situations, exact analytical solutions are difficult, time-consuming, or even impossible to find, so numerical methods provide practical and efficient alternatives. These methods are widely used in engineering, physics, economics, data science, and many other fields where accurate computation is essential for decision-making and problem-solving.

This project is designed to explore and implement a range of important numerical techniques using modern programming approaches. Its main goal is to solve non-linear equations, systems of linear equations, and optimization problems through reliable computational methods. By combining mathematical theory with programming, the project demonstrates how numerical algorithms can be transformed into useful software tools that produce fast, accurate, and organized results. In addition, it helps illustrate the importance of iteration, approximation, convergence, and error analysis in obtaining meaningful solutions.

Through this work, users can better understand how numerical methods operate in practice and how they can be applied to scientific and engineering problems. The project not only highlights the efficiency of these algorithms, but also shows the value of technology in simplifying complex calculations that would otherwise be difficult to perform manually. Overall, it serves as both a practical application and an educational model for studying the role of numerical analysis in modern computational problem-solving.

2. Objectives

- Develop a user-friendly GUI to solve complex mathematical problems.
- Implement Chapter 1 methods (Root Finding) and Chapter 2 methods (Linear Systems).
- Ensure high precision and handle mathematical errors (like division by zero).
- Provide step-by-step solutions for educational purposes.
- Display results in a clear step-by-step format to help users understand each method.

- Provide accurate numerical solutions with stopping criteria based on error percentage and maximum iterations.
- Handle invalid inputs and mathematical errors such as wrong equation format, zero determinant, and division by zero.

3. Tools & Technologies Used

1. Programming Language

- **JavaScript**: Used as the main programming language to implement all numerical methods, input handling, validation, and result generation. It was chosen because it works directly in the browser and allows the application to be interactive without requiring installation of extra software.
- **HTML**: Used to build the structure of the application, including input forms, buttons, and output sections.
- **CSS**: Used to design and style the graphical interface, making the application organized, readable, and user-friendly.

2. GUI Framework

- **No external GUI framework was used**. The interface was built using **HTML, CSS, and vanilla JavaScript**.

3. Libraries

- No external libraries were used in this project.

4. IDE

Visual Studio Code (VS Code): Used to write, edit, and manage the project files efficiently.

5. Other tools

- **Web Browser**: Used to run and test the application interface and numerical methods.
- **Built-in JavaScript Math functions**: Used for mathematical operations
- **Git**: Can be used for version control and tracking changes during development, if applied by the team.

Chapter 2 : Application User Guide

- *Open the project folder and run **index.html** in a web browser.*
- *Select the numerical method from the method list.*
- *Enter the required input values in the provided fields.*
- *For non-linear methods, type the equation and the initial guesses.*
- *For Newton-Raphson, also enter the derivative of the equation.*
- *For linear methods, enter the number of equations and write the system line by line.*
- *Enter the stopping error and maximum number of iterations when needed.*
- *Click the **Solve** button to start the calculation.*
- *View the results in the output section on the right side.*
- *Read the displayed iterations to follow the solution step by step.*
- *Check the final value shown as the required root or variable solution.*
- *Use **Clear Inputs** to remove entered values.*
- *Use **Clear Output** to remove the displayed results.*
- *If an error message appears, check the equation format and input values.*
- *Write multiplication clearly, such as **2*x** instead of **2x** for non-linear equations.*

Chapter 3 : Methods Implementation & Results

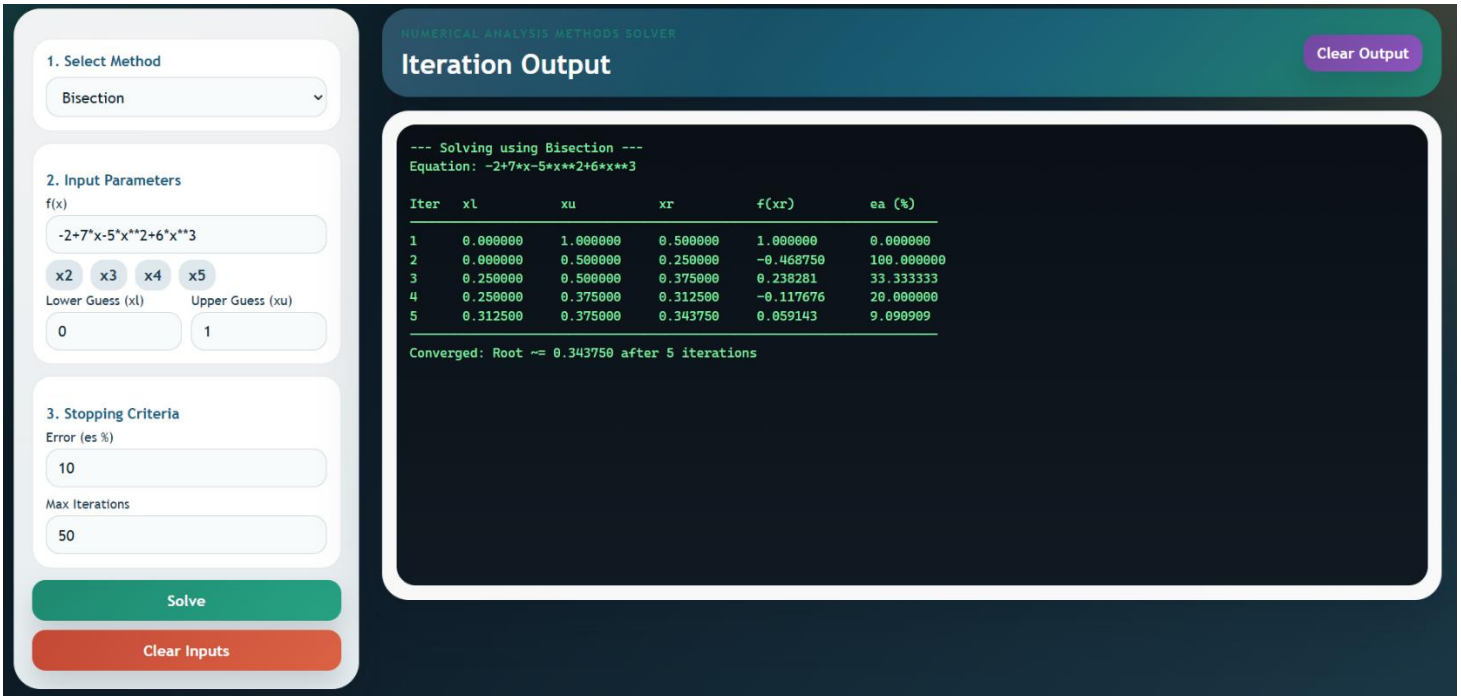
1. Chapter 1 : FINDING ROOTS OF POLYNOMIALS

1. Bisection Method

- **Code Snippet:**

```
// Solve for a root using the bisection method.
function runBisection(fn, equation, xl, xu, es, maxIter) {
  if (fn(xl) * fn(xu) >= 0) {
    log("Error: f(xl) and f(xu) must have opposite signs.");
    return;
  }

  log(`${"Iter".padEnd(6)} ${"xl".padEnd(12)} ${"xu".padEnd(12)} ${"xr".padEnd(12)} ${"f(xr)".padEnd(14)} ${"ea (%)".padEnd(10)}`);
  log("-".repeat(70));

  let xr = null;
  let xrOld = null;
  for (let i = 1; i <= maxIter; i += 1) {
    xrOld = xr;
    xr = (xl + xu) / 2;
    const fl = fn(xl);
    const fxr = fn(xr);
    const ea = xrOld !== null && xr !== 0 ? Math.abs((xr - xrOld) / xr) * 100 : 0;

    log(`${String(i).padEnd(6)} ${formatNumber(xl).padEnd(12)} ${formatNumber(xu).padEnd(12)} ${formatNumber(xr).padEnd(12)} ${formatNumber(fxr).padEnd(14)} ${formatNumber(ea).padEnd(10)}`);

    if (ea !== 0 && ea < es) {
      log("-".repeat(70));
      log(`Converged: Root ~= ${formatNumber(xr)} after ${i} iterations`);
      return;
    }

    if (fl * fxr < 0) {
      xu = xr;
    } else {
      xl = xr;
    }
  }

  log("Max iterations reached.");
}
```

- **Output (GUI Screenshot):**



2. False position Method

- **Code Snippet:**

```
// Solve for a root using the false-position method.
function runFalsePosition(fn, equation, xl, xu, es, maxIter) {
  if (fn(xl) * fn(xu) >= 0) {
    log("Error: f(xl) and f(xu) must have opposite signs.");
    return;
  }

  const xlInit = xl;
  const xuInit = xu;
  let xr = 0;
  log(`${"Iter".padEnd(8)} ${"xl".padEnd(10)} ${"xu".padEnd(10)} ${"xr".padEnd(10)} ${"f(xr)".padEnd(12)} ${"ea %".padEnd(10)}`);

  for (let i = 1; i <= maxIter; i += 1) {
    const xrOld = xr;
    const fl = fn(xl);
    const fu = fn(xu);
    xr = xu - (fu * (xl - xu)) / (fl - fu);
    const fxr = fn(xr);
    const ea = i > 1 && xr !== 0 ? Math.abs((xr - xrOld) / xr) * 100 : 0;

    log(`${String(i).padEnd(8)} ${xl.toFixed(4).padEnd(10)} ${xu.toFixed(4).padEnd(10)} ${xr.toFixed(4).padEnd(10)} ${fxr.toFixed(4).padEnd(12)} ${ea.toFixed(4).padEnd(10)}`);

    if (i > 1 && ea < es) {
      log(`Converged! Root: ${formatNumber(xr)} at ${i} iterations.`);
      return;
    }
    if (fl * fxr < 0) {
      xu = xr;
    } else {
      xl = xr;
    }
  }
  log("Reached max iterations.");
}
```

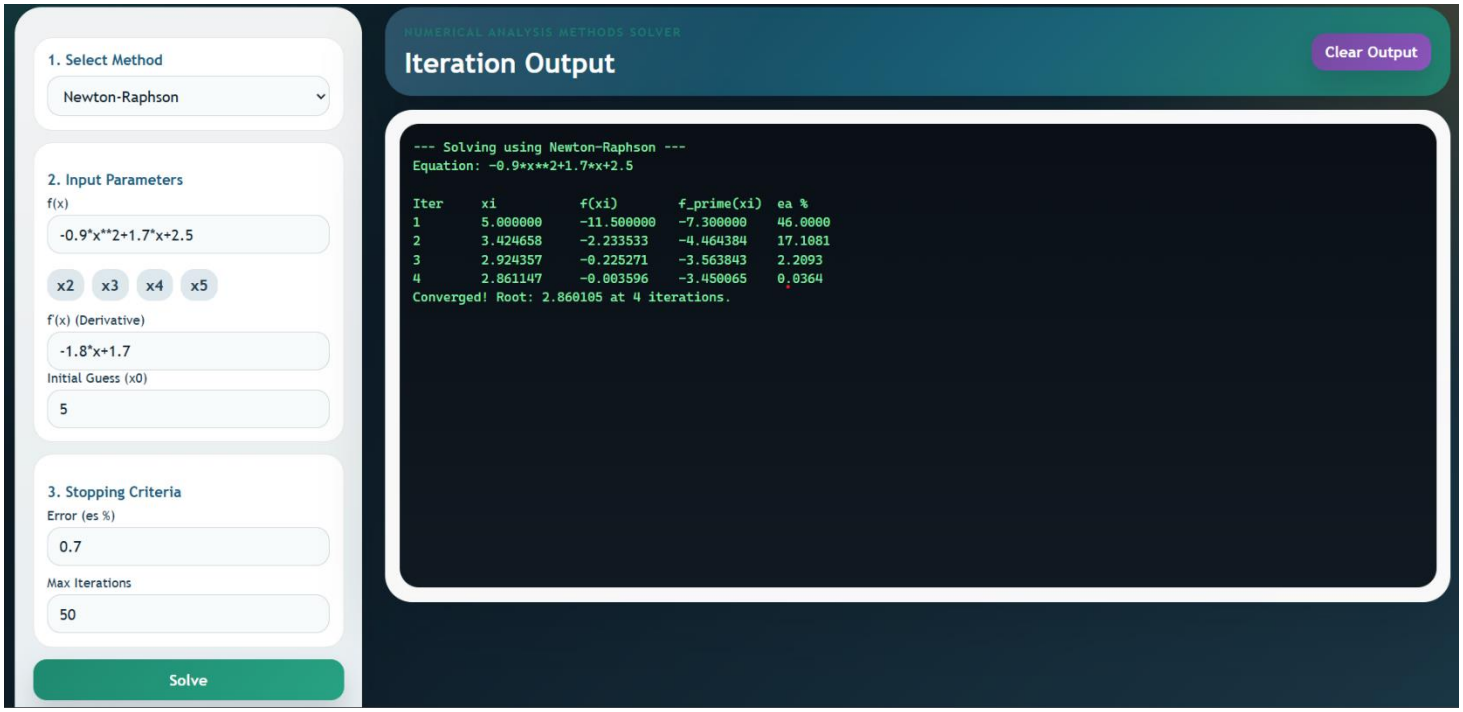
- **Output (GUI Screenshot):**

3. Newton'sMethod

- **Code Snippet:**

```
// Solve for a root using the Newton-Raphson method.
function runNewton(fn, derivative, equation, x0, es, maxIter) {
  const x0Init = x0;
  let xr = x0;
  log(`${"Iter".padEnd(8)} ${"xi".padEnd(12)} ${"f(xi)".padEnd(12)} ${"f_prime(xi)".padEnd(12)} ${"ea %".padEnd(10)}`);

  for (let i = 1; i <= maxIter; i += 1) {
    const xrOld = xr;
    const fx = fn(xrOld);
    const dfx = derivative(xrOld);

    if (dfx === 0) {
      log("Derivative is zero. Newton-Raphson fails.");
      return;
    }

    xr = xrOld - fx / dfx;
    const ea = xr !== 0 ? Math.abs((xr - xrOld) / xr) * 100 : 0;
    log(`${String(i).padEnd(8)} ${formatNumber(xrOld).padEnd(12)} ${formatNumber(fx).padEnd(12)} ${formatNumber(dfx).padEnd(12)} ${ea.toFixed(4).padEnd(10)}`);

    if (ea < es) {
      log(`Converged! Root: ${formatNumber(xr)} at ${i} iterations.`);
      return;
    }
  }

  log("Reached max iterations.");
}
```

- **Output (GUI Screenshot):**

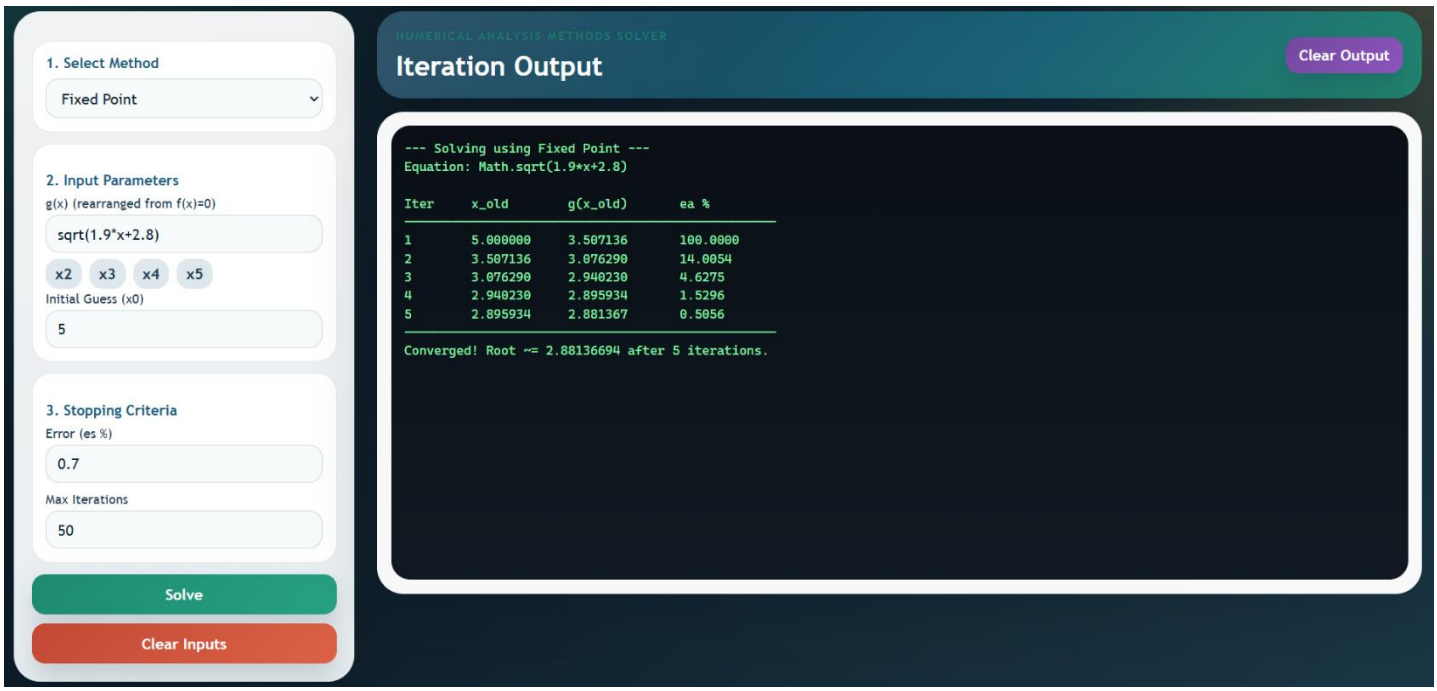


4. Simple Fixed point Method

- **Code Snippet:**

```javascript
// Solve for a root using fixed-point iteration.
function runFixedPoint(gFunction, equation, x0, es, maxIter) {
  let xr = x0;
  log(`${"Iter".padEnd(8)} ${"x_old".padEnd(12)} ${"g(x_old)".padEnd(14)} ${"ea %".padEnd(12)}`);
  log("-".repeat(50));

  for (let i = 1; i <= maxIter; i += 1) {
    const xrOld = xr;
    xr = gFunction(xrOld);

    let ea;
    if (i > 1) {
      ea = xr !== 0 ? Math.abs((xr - xrOld) / xr) * 100 : Math.abs((xr - xrOld) / (xr + 1e-10)) * 100;
    } else {
      ea = 100;
    }

    log(`${String(i).padEnd(8)} ${formatNumber(xrOld).padEnd(12)} ${formatNumber(xr).padEnd(14)} ${ea.toFixed(4).padEnd(12)}`);

    if (i > 1 && ea < es) {
      log("-".repeat(50));
      log(`Converged! Root ~= ${xr.toFixed(8)} after ${i} iterations.`);
      return;
    }

    if (i > 10 && ea > 1000) {
      log("Method diverging! Error is growing.");
      return;
    }
  }

  log("Reached max iterations.");
}
```

- **Output (GUI Screenshot):**



5. Secant Method

- **Code Snippet:**

```
// Solve for a root using the secant method.
function runSecant(fn, equation, xMinus1, x0, es, maxIter) {
  let xiMinus1 = xMinus1;
  let xi = x0;
  log(`${"Iter".padEnd(8)} ${"xi-1".padEnd(12)} ${"xi".padEnd(12)} ${"f(xi)".padEnd(12)} ${"ea %".padEnd(10)}`);

  for (let i = 1; i <= maxIter; i += 1) {
    const fXi = fn(xi);
    const fXiMinus1 = fn(xiMinus1);

    if (fXi - fXiMinus1 === 0) {
      log("Division by zero error.");
      return;
    }

    const xiPlus1 = xi - (fXi * (xi - xiMinus1)) / (fXi - fXiMinus1);
    const ea = xiPlus1 !== 0 ? Math.abs((xiPlus1 - xi) / xiPlus1) * 100 : 0;

    log(`${String(i).padEnd(8)} ${formatNumber(xiMinus1).padEnd(12)} ${formatNumber(xi).padEnd(12)} ${formatNumber(fXi).padEnd(12)} ${ea.toFixed(4).padEnd(10)}`);

    if (ea < es) {
      log(`Converged! Root: ${formatNumber(xiPlus1)} at ${i} iterations.`);
      return;
    }

    xiMinus1 = xi;
    xi = xiPlus1;
  }

  log("Reached max iterations.");
}

```

- **Output (GUI Screenshot):**

Chapter 2 : LINEAR ALGEBRAIC EQUATIONS

1. Gauss Elimination

- **Code Snippet:**

```javascript
// Solve a linear system using Gaussian elimination and back substitution.
function runGaussElimination(matrixA, vectorB, variables) {
  const n = vectorB.length;
  const augmented = matrixA.map((row, index) => [...row, vectorB[index]]);

  log("Forward Elimination:");
  for (let i = 0; i < n; i += 1) {
    let pivotRow = i;
    for (let row = i + 1; row < n; row += 1) {
      if (Math.abs(augmented[row][i]) > Math.abs(augmented[pivotRow][i])) {
        pivotRow = row;
      }
    }

    if (Math.abs(augmented[pivotRow][i]) < 1e-12) {
      throw new Error("The system has no unique solution.");
    }

    if (pivotRow !== i) {
      [augmented[i], augmented[pivotRow]] = [augmented[pivotRow], augmented[i]];
      log(`Swapped row ${i + 1} with row ${pivotRow + 1}`);
    }

    for (let row = i + 1; row < n; row += 1) {
      const factor = augmented[row][i] / augmented[i][i];
      for (let col = i; col <= n; col += 1) {
        augmented[row][col] -= factor * augmented[i][col];
      }
      augmented[row][i] = 0;
    }

    log(`After pivot step ${i + 1}:`);
    log(matrixToString(augmented));
  }
```

```
  const solution = Array(n).fill(0);
  for (let i = n - 1; i >= 0; i -= 1) {
    let rhs = augmented[i][n];
    for (let col = i + 1; col < n; col += 1) {
      rhs -= augmented[i][col] * solution[col];
    }
    solution[i] = rhs / augmented[i][i];
  }

  log("");
  log("Upper Triangular Matrix:");
  log(matrixToString(augmented.map((row) => row.slice(0, -1))));
  log("Updated Constants:");
  log(matrixToString(augmented.map((row) => row[row.length - 1])));
  log("Solution:");
  variables.forEach((variable, index) => log(`${variable} = ${formatNumber(solution[index])}`));
}

```

- **Output (GUI Screenshot):**

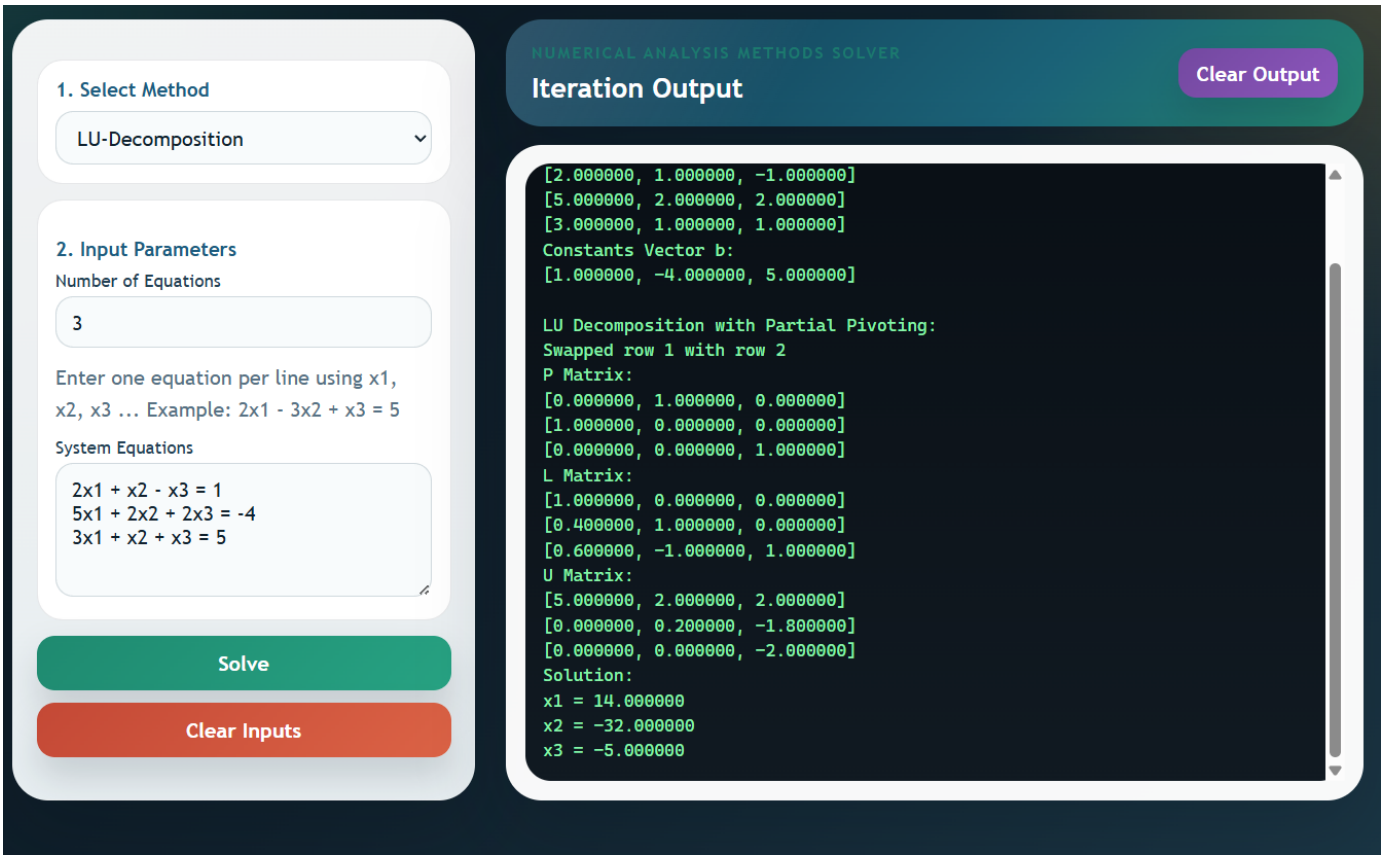
2. LU Decomposition

- **Code Snippet:**

```
// Solve a linear system using LU decomposition with partial pivoting.
function runLuPivot(matrixA, vectorB, variables) {
  const n = vectorB.length;
  const matrixL = Array.from({ length: n }, () => Array(n).fill(0));
  const matrixU = cloneMatrix(matrixA);
  const matrixP = Array.from({ length: n }, (_, row) =>
    Array.from({ length: n }, (_, col) => (row === col ? 1 : 0))
  );

  log("LU Decomposition with Partial Pivoting:");
  for (let i = 0; i < n; i += 1) {
    let pivot = i;
    for (let row = i + 1; row < n; row += 1) {
      if (Math.abs(matrixU[row][i]) > Math.abs(matrixU[pivot][i])) {
        pivot = row;
      }
    }

    if (Math.abs(matrixU[pivot][i]) < 1e-12) {
      throw new Error("The system has no unique solution.");
    }

    if (pivot !== i) {
      [matrixU[i], matrixU[pivot]] = [matrixU[pivot], matrixU[i]];
      [matrixP[i], matrixP[pivot]] = [matrixP[pivot], matrixP[i]];
      if (i > 0) {
        for (let col = 0; col < i; col += 1) {
          [matrixL[i][col], matrixL[pivot][col]] = [matrixL[pivot][col], matrixL[i][col]];
        }
      }
      log(`Swapped row ${i + 1} with row ${pivot + 1}`);
    }
```

```
    matrixL[i][i] = 1;
    for (let row = i + 1; row < n; row += 1) {
      const factor = matrixU[row][i] / matrixU[i][i];
      matrixL[row][i] = factor;
      for (let col = i; col < n; col += 1) {
        matrixU[row][col] -= factor * matrixU[i][col];
      }
      matrixU[row][i] = 0;
    }
  }

  const pb = matrixP.map((row) => row.reduce((sum, value, index) => sum + value * vectorB[index], 0));
  const y = Array(n).fill(0);
  for (let i = 0; i < n; i += 1) {
    let value = pb[i];
    for (let col = 0; col < i; col += 1) {
      value -= matrixL[i][col] * y[col];
    }
    y[i] = value;
  }

  const solution = Array(n).fill(0);
  for (let i = n - 1; i >= 0; i -= 1) {
    let value = y[i];
    for (let col = i + 1; col < n; col += 1) {
      value -= matrixU[i][col] * solution[col];
    }
    solution[i] = value / matrixU[i][i];
  }

  log("P Matrix:");
  log(matrixToString(matrixP));
  log("L Matrix:");
  log(matrixToString(matrixL));
  log("U Matrix:");
  log(matrixToString(matrixU));
  log("Solution:");
  variables.forEach((variable, index) => log(`${variable} = ${formatNumber(solution[index])}`));
}

```

- **Output (GUI Screenshot):**

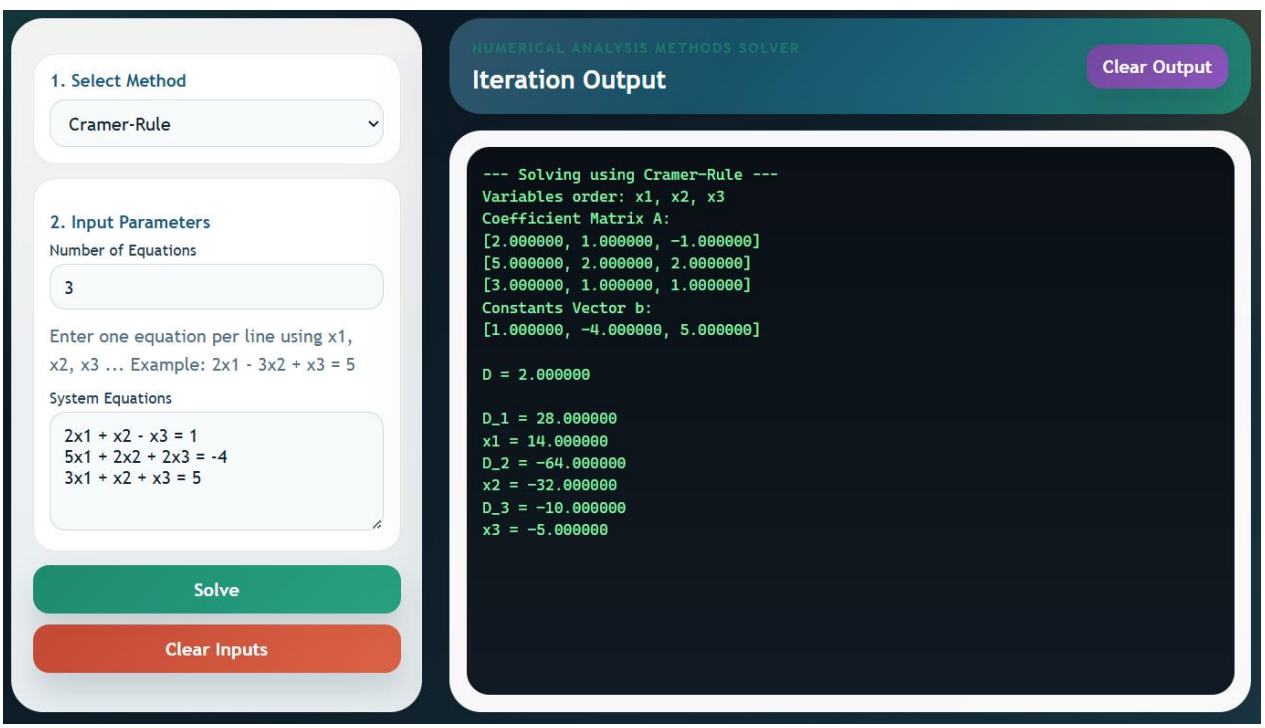


3. Cramer's Rule

- **Code Snippet:**

```javascript
// Solve a linear system using Cramer's Rule determinants.
function runCramers(matrixA, vectorB, variables) {
  const det = determinant(matrixA);
  if (Math.abs(det) < 1e-12) {
    throw new Error("Cramer's Rule requires a non-zero determinant.");
  }

  log(`D = ${formatNumber(det)}`);
  log("");

  variables.forEach((variable, index) => {
    const replaced = cloneMatrix(matrixA);
    replaced.forEach((row, rowIndex) => {
      row[index] = vectorB[rowIndex];
    });
    const detI = determinant(replaced);
    log(`D_${index + 1} = ${formatNumber(detI)}`);
    log(`${variable} = ${formatNumber(detI / det)}`);
  });
}
```

- **Output (GUI Screenshot):**



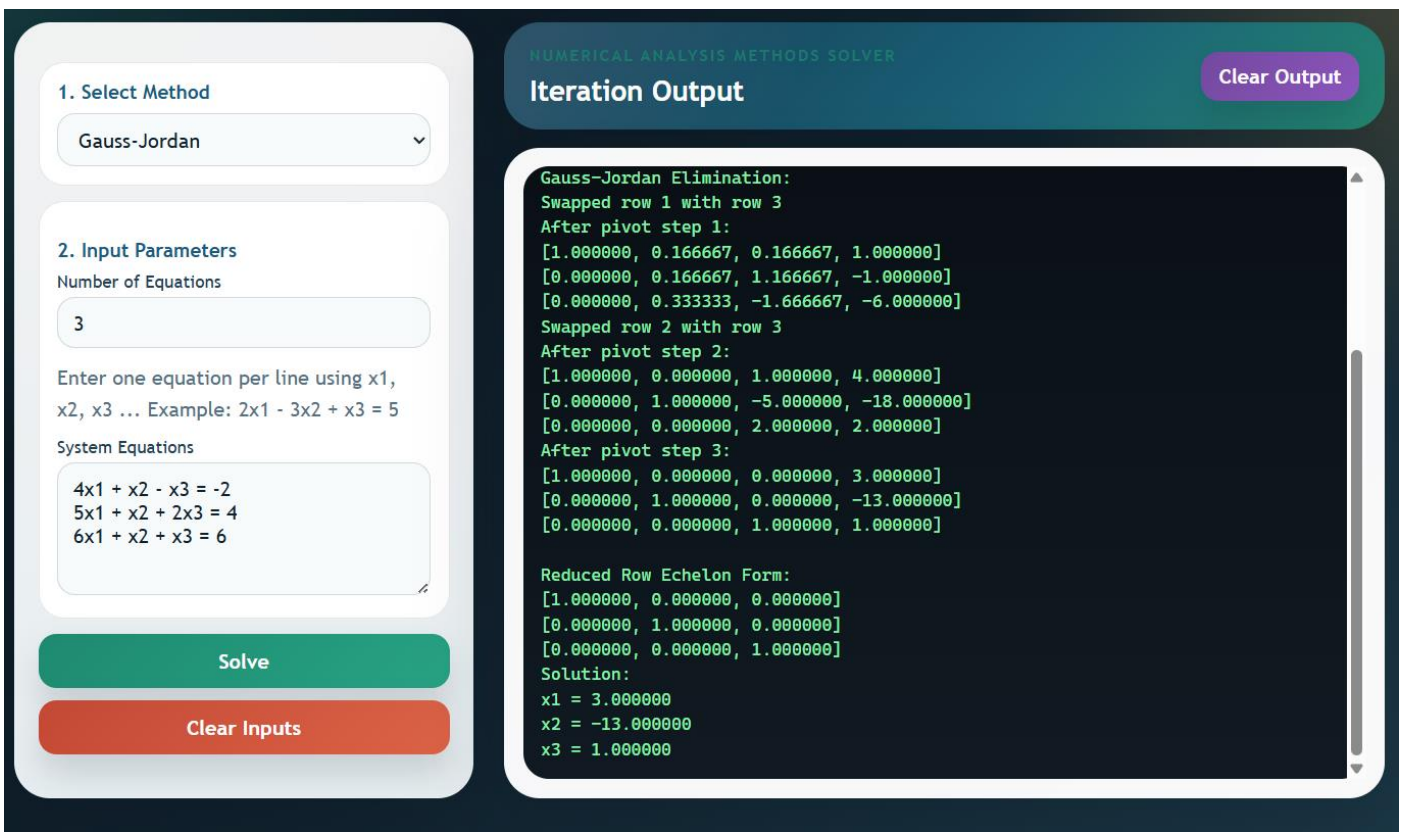
4. Gauss-Jordan

- **Code Snippet:**

```
// Solve a linear system using Gauss-Jordan elimination.
function runGaussJordan(matrixA, vectorB, variables) {
  const n = vectorB.length;
  const augmented = matrixA.map((row, index) => [...row, vectorB[index]]);

  log("Gauss-Jordan Elimination:");
  for (let i = 0; i < n; i += 1) {
    let pivotRow = i;
    for (let row = i + 1; row < n; row += 1) {
      if (Math.abs(augmented[row][i]) > Math.abs(augmented[pivotRow][i])) {
        pivotRow = row;
      }
    }

    if (Math.abs(augmented[pivotRow][i]) < 1e-12) {
      throw new Error("The system has no unique solution.");
    }

    if (pivotRow !== i) {
      [augmented[i], augmented[pivotRow]] = [augmented[pivotRow], augmented[i]];
      log(`Swapped row ${i + 1} with row ${pivotRow + 1}`);
    }

    const pivot = augmented[i][i];
    for (let col = i; col <= n; col += 1) {
      augmented[i][col] /= pivot;
    }

    for (let row = 0; row < n; row += 1) {
      if (row === i) {
        continue;
      }
      const factor = augmented[row][i];
      for (let col = i; col <= n; col += 1) {
        augmented[row][col] -= factor * augmented[i][col];
      }
      augmented[row][i] = 0;
    }
```

```
      log(`After pivot step ${i + 1}:`);
      log(matrixToString(augmented));
    }

    log("");
    log("Reduced Row Echelon Form:");
    log(matrixToString(augmented.map((row) => row.slice(0, -1))));
    log("Solution:");
    variables.forEach((variable, index) => log(`${variable} = ${formatNumber(augmented[index][augmented[index].length - 1])}`));
  }
```

- **Output (GUI Screenshot):**



Summary & Conclusion

In this project, we developed a numerical analysis application that solves both non-linear equations and systems of linear equations using a simple and interactive interface. The application includes important numerical methods such as Bisection, False Position, Fixed Point, Newton-Raphson, Secant, Gauss Elimination, Gauss-Jordan, Cramer's Rule, and LU Decomposition. It was designed to display step-by-step results, making it useful not only for solving problems but also for understanding how each method works in practice.

Through this project, we learned how numerical methods can be applied using programming to solve mathematical problems efficiently and accurately. We also gained experience in handling user input, validating equations, managing mathematical errors, and presenting results clearly. In addition, the project helped strengthen our understanding of approximation, convergence, iteration, and matrix operations. Overall, this work showed the importance of combining mathematics with programming to create practical tools for education and problem-solving.